

TRABAJO PRÁCTICO N° 3

"Mi Primera SPA Reactiva"

Objetivo General: Crear una **Single Page Application (SPA)** funcional utilizando **Vue.js 3** (cargado por CDN) sobre un tema propuesto por el estudiante.

El desafío es doble:

1. Simular la navegación entre "páginas" **sin usar Vue Router** (con la técnica de v-show).
2. Demostrar el uso de la **reactividad "en dos sentidos"** de Vue mediante el uso obligatorio de sus directivas principales.

Requisitos Técnicos

1. Estructura del Proyecto

El proyecto debe estar organizado de forma similar al ejemplo CocinaDeJuan.html (adjunto en la aula virtual):

- index.html:
 - Contendrá el DIV principal (<div id="app">...</div>).
 - Contendrá el menú de navegación (los <button>).
 - Contendrá los div que se mostrarán u ocultarán (ej: <div v-show="vistaInicio">...</div>).
 - Cargará los scripts (Vue CDN, componentes.js, script.js).
- componentes.js:
 - Donde se definirán y registrarán **globalmente** todos los componentes de Vue (ej: app.component('mi-header', { ... })).
- script.js:
 - Donde se creará la instancia principal de Vue (Vue.createApp({ ... }).mount('#app')).
 - Contendrá la lógica de "navegación" y todos los **datos reactivos** (el "estado").

2. Simulación de Navegación (con v-show)

La aplicación debe tener un **mínimo de 4 "vistas"** (ej: Inicio, Catálogo, Contacto).

1. **Estado de Visibilidad:** En script.js, la instancia de Vue debe tener propiedades en data() para controlar qué vista está activa (ej: vista_inicio: true, vista_catologo: false, etc.).
2. **Método de Navegación:** Debe existir un método (ej: mostrarVista(vista)) que ponga todas las variables de vista en false y active la correcta en true.
3. **Botones y Vistas:** En index.html, los botones del menú deben usar @click para llamar a mostrarVista(), y los div de las vistas deben usar v-show para reaccionar al

estado.

3. Requisitos de Reactividad y Directivas

El contenido *dentro* de las vistas debe ser dinámico. Es **obligatorio** el uso de las siguientes directivas en la aplicación:

- **v-for:**
 - **Requisito:** Debes tener un array en data() (en script.js) con al menos 3 elementos (ej: lista de proyectos, lista de productos, lista de recetas).
 - **Requisito:** Debes renderizar (dibujar) este array en el HTML usando la directiva v-for. (Ej: `<ul> <li v-for="item in miArray">{{ item.nombre }}</li> </ul>`).
- **v-model (Reactividad "en dos sentidos"):**
 - **Requisito:** Debes tener al menos una vista (ej: "Contacto" o "Agregar Item") con un formulario (`<form>`).
 - **Requisito:** Este formulario debe usar v-model en sus campos (`<input>`, `<textarea>`) para enlazar lo que escribe el usuario a una propiedad en data().
 - **Requisito:** Debes mostrar en la misma página (en algún lugar) el valor de esa propiedad (ej: `<p>Estás escribiendo: {{ mensajeDelFormulario }}</p>`) para demostrar que el enlace es en tiempo real.
- **v-bind :**
 - **Requisito:** Debes usar v-bind (o su atajo :) para controlar un atributo HTML desde tus datos.
 - **Ejemplos:** v-bind:src (o :src) para la URL de una imagen, v-bind:href (o :href) para un enlace, o v-bind:class para una clase CSS dinámica.
- **v-on (o @) (Manejo de Eventos):**
 - **Requisito:** Además de usar @click para la navegación, debes usarlo para al menos una acción más.
 - **Ejemplos:** Un botón "Guardar" en tu formulario v-model, un botón "Me Gusta" en un item de v-for, o un @submit en la etiqueta `<form>`.
- **v-if / v-else (Renderizado Condicional):**
 - **Requisito:** Debes usar v-if (distinto del v-show de navegación) para mostrar u ocultar algo basándote en una condición.
 - **Ejemplos:** Mostrar un mensaje "El campo está vacío" si el v-model no tiene texto, o mostrar "No hay productos en la lista" si el array de v-for está vacío.

4. Componentización (en componentes.js)

- **Requisito Mínimo:** Crear al menos **4 componentes** globales:
 1. Un componente de cabecera (`<mi-header></mi-header>`).
 2. Un componente de pie de página (`<mi-footer></mi-footer>`).
 3. Al menos dos (2) componentes para el contenido de las vistas (ej: `<vista-inicio></vista-inicio>`, `<vista-catalogo></vista-catalogo>`).

- **Props:** Al menos uno de los componentes debe recibir datos a través de props (ej. `<mi-tarjeta :titulo="item.titulo"></mi-tarjeta>`).

 Ideas de Temas (Inspiración)

Tema	Vistas / Componentes sugeridos	Datos Reactivos (para v-for / v-model)
Portafolio Profesional	inicio (bio), proyectos (galería), contacto (form)	array de proyectos, objeto para el form de contacto.
Recetario Simple	inicio (bienvenida), recetas (lista), agregar (form)	array de recetas, objeto para la nueva receta.
Catálogo de Hobbies	inicio (intro), coleccion (lista), contacto	array de items de la colección, objeto para el form.
Web de un Restaurante	inicio (bienvenida), menu (lista), reservas (form)	array de platos del menú, objeto para la reserva.

 Criterios de Evaluación

- **Funcionalidad SPA:** ¿La navegación entre vistas funciona con v-show sin recargar la página?
- **Reactividad y Directivas:** ¿Se utilizaron **todas** las directivas solicitadas (v-for, v-model, v-bind, v-on, v-if) de forma correcta?
- **Componentización:** ¿El código está modularizado en componentes (definidos en componentes.js)?
- **Estado:** ¿Se manejaron correctamente los datos (el "estado") en la instancia raíz de Vue (en script.js)?
- **Propuesta de Tema:** ¿La estructura de la aplicación es coherente con el tema elegido?
- Es una **propuesta superadora?**:
 - **Directivas:** Se usan otras directivas adicionales?
 - **Vistas:** Propuso una app con mas de 4 vistas?
 - **Reactividad en dos sentidos:** Los componentes propuestos manejan directivas que implican reactividad en doble sentido?